FEROZE GANDHI INSTITUTE OF ENGINEERING AND TECHNOLOGY

Raebareli, Uttar Pradesh

Department of Computer Science and Engineering

B.TECH FINAL YEAR PROJECT REPORT

Academic Year: 2024 - 2025

PROJECT TITLE

VideoRAG: A Timestamp-Grounded Metadata-Augmented Retrieval System for Educational Video Understanding

Submitted By

Garima Singh

Ankita Yadav

Anushka Srivastava

Under the Guidance of

[Dr.Suni kumar Verma,Computer Science and Engineering]

In partial fulfillment of the requirements for the degree of

Bachelor of Technology (B.Tech)

in Computer Science and Engineering

Feroze Gandhi Institute of Engineering and Technology

Raebareli, Uttar Pradesh | AKTU Affiliated

Certificate

This is to certify that the project report entitled:

"VideoRAG: A Timestamp-Grounded Metadata-Augmented Retrieval System for Educational Video Understanding"

has been submitted by:

Garima Singh | Ankita Yadav | Anushka Srivastava

in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from Feroze Gandhi Institute of Engineering and Technology, Raebareli, affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU), Lucknow.

The work embodied in this project has been carried out under our supervision and to our satisfaction. It is an original work and has not been submitted earlier for any degree, diploma, or similar award to any university or institution.

Project Guide

[Dr.Sunil kumar verma]

Dept. of CSE

FGIET, Raebareli

Head of Department

[Dr.Sunil kumar verma]

Dept. of CSE

FGIET, Raebareli

Declaration

We, the undersigned, students of Bachelor of Technology (Computer Science and Engineering) at Feroze Gandhi Institute of Engineering and Technology, Raebareli, do hereby declare that the project report entitled:

"VideoRAG: A Timestamp-Grounded Metadata-Augmented Retrieval System for Educational Video Understanding"

is our original work and is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology. This work has not been previously submitted to any other university or institution for the award of any degree, diploma, certificate, or any other academic honor.

We further declare that we have properly acknowledged and cited all the sources of information used in this project. The content of this report accurately represents the work we have done during the course of this project.

Garima Singh	**Ankita Yadav**	**Anushka Srivastava**
Roll No: 2201870100059	Roll No: 2201870100019	Roll No: 2201870100029

Date: _____ Place: Raebareli

Table of Contents

List of Figures

List of Tables

Chapter 1: Introduction

1.1 Background and Motivation

The proliferation of online educational platforms and Massive Open Online Courses (MOOCs) has transformed how students access knowledge. Video-based learning has become one of the most dominant modalities, with platforms such as YouTube, NPTEL, Coursera, and institutional LMS repositories hosting thousands of hours of lecture recordings. Despite this abundance of content, students still face significant challenges in extracting specific knowledge from video lectures efficiently.

When a student watches a two-hour recorded lecture and needs to revisit a specific concept explained at the 47-minute mark, they are left with few options: scrub manually through the video, rely on imprecise auto-generated captions, or simply re-watch entire sections. This inefficiency becomes compounding across a full semester of multiple courses, severely impacting study productivity.

Intelligent Tutoring Systems (ITS) have long been proposed as a solution for personalized, on-demand learning assistance. The emergence of Large Language Models (LLMs) such as GPT-4, LLaMA, and Gemini has dramatically improved the natural language understanding and generation capabilities available to such systems. However, a standalone LLM suffers from a critical weakness: hallucination. When asked a question outside its training distribution, or requiring precise factual recall, an LLM tends to generate plausible-sounding but incorrect responses.

Retrieval-Augmented Generation (RAG) was proposed as a solution to this problem — by grounding LLM responses in retrieved, verified context from a knowledge base, the system can dramatically reduce hallucination and improve factual accuracy. Standard RAG systems applied to educational content, however, suffer from their own set of issues: they retrieve isolated text snippets without temporal context, lack any mechanism to link answers back to source video material, and fail to aggregate sufficient context for nuanced educational explanations.

Knowledge Graph-enhanced RAG (KG-RAG), as demonstrated by Dong et al. (2025), improves upon standard RAG by structuring course knowledge into an explicit graph of entity relationships, enabling multi-hop retrieval that follows conceptual connections. While effective, KG-RAG requires significant upfront investment: expert

domain knowledge is needed to validate extracted entity-relationship triples, and graph construction must be redone for each new course or domain.

This project proposes VideoRAG — a practical, scalable alternative that achieves improved retrieval quality and temporal grounding without the complexity of a full knowledge graph. By combining intelligent chunk aggregation, metadata-enriched embeddings, and strict prompt engineering, VideoRAG delivers timestamp-linked, hallucination-controlled answers directly from lecture video content.

1.2 Problem Statement

Existing AI tutoring systems suffer from several interrelated problems when deployed in video-based educational contexts:

- Fragmented Context Retrieval: Standard RAG systems divide text into small chunks for embedding efficiency. While this improves individual chunk precision, it means that retrieved context often captures only a narrow slice of a broader explanation. When a lecturer spends three minutes explaining a concept, retrieving a single 30-second chunk provides an incomplete picture.
- No Temporal Grounding: Existing RAG systems retrieve text without any reference to where in the source material that text appears. A student cannot identify which portion of a video explains the concept they asked about, forcing manual navigation through long recordings.
- Hallucination in LLM Responses: Without strict prompt engineering, LLMs generating responses from retrieved context often fill gaps in that context with plausible but unverified information. In an educational setting, this can introduce factual errors that mislead students.
- High Complexity of Graph-Based Systems: KG-RAG and similar approaches require structured knowledge graph construction, expert validation of entity-relationship triples, and significant infrastructure investment. This makes them impractical for deployment across varied courses or institutions with limited resources.
- Limited Scalability: Graph-based systems do not scale gracefully to large video repositories. Adding new course content requires re-running extraction pipelines and re-validating graph edges.

These problems collectively result in AI tutoring systems that are either insufficiently accurate, disconnected from source material, too complex to deploy, or too limited in scale to serve diverse educational needs.

1.3 Proposed Solution

VideoRAG addresses these problems through a multi-stage pipeline that combines the following key innovations:

- Timestamp-Preserving Transcription: Using OpenAI Whisper, lecture audio is transcribed with word-level timestamps, enabling every piece of retrieved text to be linked back to an exact position in the source

video.

- Intelligent Chunk Aggregation: Rather than retrieving isolated small chunks, VideoRAG merges consecutive chunks (default: 5 per merged block) to provide richer context windows to the LLM, improving the completeness and coherence of generated answers.
- Metadata-Enriched Embeddings: Each chunk is embedded with combined title and content metadata, improving semantic matching quality when queries are received.
- Hallucination-Controlled Prompt Engineering: A strictly designed prompt instructs the LLM to answer only from provided context, never guessing or extrapolating beyond the retrieved information.
- Direct Video Navigation: Every answer includes the start and end timestamp of the source chunk, allowing students to navigate directly to the relevant portion of the lecture video.

1.4 Objectives

The primary objectives of this project are:

1. To design and implement a complete end-to-end pipeline for converting raw lecture videos into a queryable knowledge base with temporal metadata.
2. To develop an intelligent chunking and merging strategy that provides semantically coherent context windows for LLM response generation.
3. To implement a metadata-augmented embedding system using BGE-M3 that improves retrieval accuracy over standard embedding approaches.
4. To build a Streamlit-based user interface that provides students with accurate, timestamp-grounded answers to their queries.
5. To evaluate the system against traditional RAG approaches and demonstrate improvements in retrieval accuracy, context completeness, and answer quality.
6. To provide a scalable, deployable system architecture that can be applied across diverse educational courses without significant reconfiguration.

1.5 Scope of the Project

The scope of this project encompasses the complete pipeline from raw video input to user-facing question-answering, including:

- Video processing and audio extraction using FFmpeg
- Automatic speech recognition and timestamp extraction using Whisper
- Text preprocessing, chunking, and chunk merging
- Vector embedding generation using BGE-M3
- Semantic retrieval using cosine similarity
- LLM-based answer generation with LLaMA 3.2 (local) and GPT-5 (API fallback)
- Streamlit-based interactive user interface

The project does not include: live video streaming support, user authentication and session management, multi-user deployment architecture, or knowledge graph construction. These are identified as future work directions.

1.6 Report Organization

The remainder of this report is organized as follows:

- Chapter 2 provides a review of related work in RAG systems, Intelligent Tutoring Systems, and Knowledge Graph-based approaches.
- Chapter 3 describes the theoretical background and foundational concepts underlying the proposed system.
- Chapter 4 presents the detailed system design, architecture, and module descriptions.
- Chapter 5 provides the implementation details, including data structures, algorithms, and code structure.
- Chapter 6 presents the testing methodology and results, including comparison with baseline systems.
- Chapter 7 discusses the results, system limitations, and directions for future work.
- Chapter 8 concludes the report.

Chapter 2: Literature Review

2.1 Intelligent Tutoring Systems

Intelligent Tutoring Systems (ITS) have a long history in educational technology research, dating back to the 1970s with systems such as SOPHIE and SCHOLAR. Modern ITS leverage machine learning and natural language processing to provide adaptive, personalized learning experiences. Gromyko et al. (2017) demonstrated that AI-based tutoring systems can effectively serve as learning partners for human intellect, providing pedagogically sound interactions. Kasneci et al. (2023) provided a comprehensive analysis of the opportunities and challenges that large language models bring to education, highlighting both the transformative potential and the risks of hallucination and bias.

Baumgart and Madany Mamlouk (2022) proposed a knowledge model for AI-driven tutoring systems that emphasized the importance of structured domain representation. Their work highlighted that pure language model approaches, while linguistically fluent, lack the structured understanding of domain concepts needed for effective pedagogical interaction. This insight motivates the knowledge-augmented retrieval approach adopted in VideoRAG.

Moore et al. (2023) investigated the use of generative LLMs as the next-generation interface for educational content generation, identifying key challenges in maintaining factual accuracy and curriculum alignment. Their findings confirm that standalone LLMs are insufficient for educational deployment and must be augmented with domain-specific knowledge retrieval.

2.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation was formally introduced by Lewis et al. (2020) as a framework for enhancing LLM factual accuracy by combining parametric knowledge (stored in model weights) with non-parametric retrieval from external document stores. The RAG framework embeds documents into a shared vector space alongside queries, retrieves top-k most similar documents using approximate nearest neighbor search, and conditions LLM generation on the retrieved context.

Cai et al. (2022) provided a comprehensive survey of recent advances in retrieval-augmented text generation, cataloguing various architectural approaches including sparse retrieval (BM25), dense retrieval (DPR, ANCE),

and hybrid methods. Their survey identified the limitation of isolated chunk retrieval as a persistent weakness — when documents are split into small chunks for efficient indexing, the retrieved context often lacks the broader explanatory context needed for educational use cases.

Sarmah et al. (2024) proposed HybridRAG, which integrates knowledge graphs with vector retrieval for efficient information extraction in financial document analysis. Their system demonstrated that combining structural and semantic retrieval can significantly improve answer completeness. VideoRAG is inspired by this hybrid philosophy but focuses on practical simplicity and video-specific temporal grounding rather than full graph construction.

Lee (2024) specifically investigated the development of computer-based tutoring systems using Generative AI and RAG, confirming that RAG-based tutors significantly outperform standalone LLMs in educational settings but identifying the lack of source navigation and temporal grounding as critical gaps for video-based learning.

2.3 Knowledge Graph-Enhanced RAG (KG-RAG)

Dong et al. (2025) introduced KG-RAG specifically in the context of AI tutoring for university courses. Their system constructs an explicit knowledge graph from course materials by extracting entity-relationship triples using DeepSeek-V3, validated by a domain expert. A Knowledge-Guided Retrieval (KGR) mechanism then traverses this graph to retrieve expanded context — when a student asks about 'mortgage-backed securities,' the system not only retrieves directly related content but also traverses graph edges to incorporate related concepts like 'collateralized debt obligations' and 'subprime crisis.'

Their controlled experiment with 76 university students demonstrated a 35% improvement in assessment scores ($p < 0.001$, Cohen's $d = 0.86$) for KG-RAG students compared to standard RAG students. This significant result validates the core insight that structured relational context improves learning outcomes.

However, their approach requires expert annotation for knowledge graph validation, making it impractical for rapid deployment across diverse courses. Bui et al. (2024) investigated cross-data knowledge graph construction for LLM-enabled educational question answering, confirming that while KG-based approaches are powerful, their construction overhead is a significant barrier to adoption. VideoRAG directly addresses this limitation by achieving improved context quality through chunk aggregation rather than graph construction.

2.4 Speech Recognition and Video Processing

OpenAI's Whisper (2022) represents the current state of the art in automatic speech recognition, trained on 680,000 hours of multilingual speech data. Whisper's key advantage for this project is its ability to generate word-level and segment-level timestamps alongside transcriptions, enabling precise temporal mapping

between text content and video positions. Its robustness to accents, background noise, and varying recording quality makes it particularly suitable for processing real-world lecture recordings.

FFmpeg is the industry-standard multimedia processing framework, providing reliable audio extraction from diverse video formats (MP4, MKV, AVI, MOV) with minimal quality loss. Its cross-platform compatibility and command-line interface make it straightforward to integrate into Python-based processing pipelines.

2.5 Embedding Models for Semantic Retrieval

The BAAI General Embedding model (BGE-M3, 2024) provides state-of-the-art multilingual sentence embeddings capable of representing complex semantic relationships in high-dimensional vector spaces. BGE-M3's architecture supports multi-granularity retrieval, enabling effective matching between user queries and educational content at both the sentence and paragraph levels. When served via Ollama for local inference, BGE-M3 provides high-quality embeddings without API costs or data privacy concerns.

Devlin et al.'s BERT (2018) established the foundational architecture for contextual word embeddings that subsequent models including BGE-M3 build upon. The key insight that context fundamentally changes word meaning — and therefore that embeddings must be context-sensitive — directly motivates the use of neural embedding models over simpler bag-of-words approaches for educational content retrieval.

2.6 Research Gap and Contribution

The reviewed literature reveals a clear research gap: while sophisticated RAG and KG-based systems have demonstrated strong performance in educational settings, none specifically address the challenge of temporal grounding in video-based learning. Existing systems treat educational content as static text, losing the temporal structure that is inherent to and valuable in video lectures. VideoRAG fills this gap by:

- Treating video timestamp as a first-class metadata attribute throughout the entire pipeline
- Providing a practical, deployable alternative to knowledge graph construction that achieves improved context quality through intelligent aggregation
- Delivering a complete, working system rather than a theoretical framework, enabling direct deployment in educational institutions

Chapter 3: Theoretical Background

3.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a framework for augmenting large language model responses with externally retrieved information. The standard RAG pipeline operates as follows:

Given a user query q and a knowledge base KB = {d_1, d_2, ..., d_n}, the system first computes embeddings for all documents and the query. Let e_q = Embed(q) and E = {Embed(d_i) | d_i ∈ KB}. Similarity scores are computed as S = {cos(e_q, e_{d_i}) | d_i ∈ KB}, where cosine similarity is defined as:

$$\cos(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

The top-k documents with highest similarity are retrieved: I_top = argmax_k(S). These documents are concatenated into context c = concatenate({d_i | i ∈ I_top}), and the LLM generates a response:

$$r_{RAG} = LLM(c, q)$$

VideoRAG extends this standard formulation by enriching documents with temporal metadata, implementing a chunk aggregation strategy that increases effective context window size, and applying strict prompting constraints on the LLM generation step.

3.2 Automatic Speech Recognition with Whisper

OpenAI Whisper is a transformer-based automatic speech recognition (ASR) system trained using a supervised learning approach on a large-scale weakly labelled dataset of 680,000 hours of multilingual audio. Whisper's architecture consists of a convolutional neural network encoder that processes log-Mel spectrograms of audio input, and an autoregressive transformer decoder that generates text tokens.

The critical feature of Whisper for this project is its ability to generate segment-level timestamps alongside transcribed text. Each segment s_i is represented as a tuple:

$$s_i = (t_{start_i}, t_{end_i}, text_i)$$

where t_start_i and t_end_i are the start and end times in seconds, and text_i is the transcribed text for that segment. This temporal metadata is preserved and propagated through all subsequent processing stages in VideoRAG.

3.3 Text Chunking and Context Aggregation

A fundamental design decision in RAG systems is the choice of chunk size. Small chunks (50-100 tokens) provide precise retrieval granularity but may lack sufficient context for comprehensive answers. Large chunks (500-1000 tokens) provide richer context but may dilute retrieval precision by including irrelevant content.

VideoRAG adopts a two-stage strategy: first, text is divided into manageable chunks of approximately 200-300 tokens, preserving natural sentence boundaries and timestamp alignment. Then, the merge_chunks.py module aggregates N consecutive chunks (where N = 5 by default) into merged blocks. A merged block M_j covering chunks c_{j*N} through c_{j*N + N-1} is defined as:

$$M_j = \{ \text{text: concat}(c_{j*N}...c_{j*N+N-1}), \text{t\_start: } c_{j*N}.t_{start}, \text{t\_end: } c_{j*N+N-1}.t_{end} \}$$

This aggregation increases the effective context size provided to the LLM while maintaining precise timestamp boundaries. A student receiving an answer from M_j knows exactly which time range of the video contains the relevant explanation.

3.4 Dense Vector Embeddings

The BGE-M3 embedding model maps text sequences to dense vectors in a high-dimensional space (typically 1024 dimensions) such that semantically similar texts are mapped to geometrically proximate vectors. This property enables efficient semantic search: a query about 'what is a DataFrame?' will have high cosine similarity with chunks discussing pandas DataFrames, even if the exact phrasing differs.

In VideoRAG, embeddings are computed for the concatenation of chunk title and content: embed_i = BGE-M3(title_i + " " + content_i). This metadata-enrichment improves embedding quality for educational content

where section titles and topic labels provide important disambiguation context.

3.5 Prompt Engineering for Hallucination Control

Hallucination in LLMs arises when the model generates text that fills gaps in its context with plausible-sounding but unverified information. In educational applications, hallucination is particularly harmful as students may accept incorrect information as factual.

VideoRAG employs strict prompt engineering to prevent hallucination. The LLM receives the following instruction template:

```
You are an educational assistant. Answer the student's question ONLY using the provided context. If the answer cannot be found in the context, state that clearly. Do not guess or extrapolate. Always reference the timestamp where the answer can be found.
```

This instruction template serves three purposes: it constrains the LLM to the retrieved context, it instructs honest uncertainty expression when information is absent, and it mandates timestamp inclusion in responses for direct video navigation.

Chapter 4: System Design and Architecture

4.1 Overall System Architecture

VideoRAG is designed as a modular pipeline architecture, where each stage performs a well-defined transformation on the data and passes the result to the next stage. This design promotes maintainability, testability, and extensibility. The overall system follows the pipeline:

Video → Audio → Speech-to-Text → JSON → Chunking → Merging → Embedding → Retrieval → LLM → UI

Each arrow in this pipeline represents a well-defined data transformation with a specified input format, output format, and processing logic. Table 1 provides a detailed description of each stage.

Table 1: System Pipeline Stages

Step	Stage	Description
Step 1	**Video Input**	Raw lecture videos uploaded to the system
Step 2	**Audio Extraction**	FFmpeg extracts audio stream from the video
Step 3	**Transcription**	Whisper ASR converts audio to timestamped text
Step 4	**Preprocessing**	JSON transcripts cleaned and normalized
Step 5	**Chunking**	Text divided into overlapping semantic chunks
Step 6	**Chunk Merging**	Consecutive chunks merged for context continuity
Step 7	**Embedding**	BGE-M3 model converts chunks into vector embeddings
Step 8	**Retrieval**	Cosine similarity matches query to top-k chunks
Step 9	**LLM Generation**	LLaMA 3.2 / GPT fallback generates final answer
Step 10	**UI Output**	Streamlit displays answer with video timestamps

4.2 Data Flow Architecture

Data flows through the VideoRAG system in a series of transformations, each adding semantic richness and structure to the raw video content. The fundamental data unit at each stage is:

- Stage 1 (Input): Raw video file (MP4, MKV, AVI, MOV formats supported)
- Stage 2 (Audio): Extracted WAV/MP3 audio stream
- Stage 3 (Transcript): JSON array of Whisper segments, each with {start, end, text} fields

- Stage 4 (Preprocessed): Cleaned and normalized JSON transcript with validated timestamps
- Stage 5 (Chunks): JSON array of chunk objects: {id, text, start_time, end_time, title}
- Stage 6 (Merged Chunks): JSON array of merged chunk objects: {id, text, start_time, end_time, title, source_chunks}
- Stage 7 (Embeddings): JSON array with added 'embedding' field containing 1024-dimensional float vector
- Stage 8 (Retrieved): Subset of top-k merged chunks ranked by cosine similarity to query
- Stage 9 (Response): Natural language answer string with embedded timestamp references

4.3 Module Design

The system is implemented as five Python modules, each encapsulating a distinct stage of the pipeline. Table 4 describes the module-wise functionality.

Table 4: Module-wise Functionality Description

Module / File	Functionality
preprocess_json.py	Cleans raw Whisper JSON transcripts; normalizes text, removes noise, validates segment timestamps
create_chunks.py	Divides transcript text into fixed-size overlapping semantic chunks with associated start/end timestamps
merge_chunks.py	Merges consecutive N chunks (default: 5) into aggregated context blocks; preserves time boundaries
process_incoming.py	Orchestrates the full preprocessing pipeline; monitors input directory and triggers processing
app.py	Streamlit-based main application; handles user query input, embedding, retrieval, LLM call, and UI display

4.3.1 preprocess_json.py

The preprocessing module takes as input the raw JSON output from Whisper and produces a cleaned, validated transcript. Key processing steps include:

- Removal of filler words and transcription artifacts (um, uh, [inaudible], etc.)
- Normalization of text encoding and character sets
- Validation of timestamp monotonicity — each segment's end time must be greater than its start time
- Merging of very short segments (< 2 seconds) that may arise from Whisper's segmentation on pauses
- Removal of duplicate or near-duplicate segments that occasionally appear in Whisper output for music or background audio

4.3.2 create_chunks.py

The chunking module divides the preprocessed transcript into semantically coherent chunks suitable for embedding. The algorithm:

1. Iterates through transcript segments sequentially
2. Accumulates segments into a current chunk until the chunk word count exceeds a threshold (default: 150 words)
3. When the threshold is reached, finalizes the current chunk with the timestamp of its first and last included segment
4. Assigns each chunk a sequential ID and auto-generates a title from the first meaningful noun phrase in the chunk text
5. Outputs the chunk array as a JSON file

This approach ensures that chunk boundaries respect natural speech boundaries (segment breaks from Whisper) rather than arbitrarily cutting mid-sentence, which would degrade semantic coherence.

4.3.3 merge_chunks.py

The merging module is the core innovation that addresses fragmented context retrieval. Rather than storing and retrieving individual small chunks, this module creates larger context windows by merging N consecutive chunks. The merging algorithm:

1. Groups chunks into windows of size N (default: N = 5)
2. For each window, concatenates the text of all N chunks in order
3. Records the start_time of the first chunk and the end_time of the last chunk as the merged block's timestamps
4. Optionally stores references to source chunk IDs for traceability
5. Outputs merged blocks as a new JSON file

This approach ensures that when the retrieval system identifies a relevant merged block, the LLM receives approximately 750-1000 words of continuous, contextually coherent educational content — comparable to a full lecture segment — rather than a short isolated snippet.

4.3.4 process_incoming.py

The pipeline orchestration module provides a unified entry point for processing new video content. It monitors a designated input directory, detects new video files, and sequentially invokes the processing stages: FFmpeg audio extraction, Whisper transcription, preprocessing, chunking, merging, and embedding generation. This module enables batch processing of multiple lecture videos and provides logging and error handling for production deployment.

4.3.5 app.py

The main application module implements the Streamlit-based user interface and the query processing logic. On startup, it loads the precomputed chunk embeddings from disk. When a user submits a query:

1. The query is embedded using BGE-M3 via Ollama API
2. Cosine similarity is computed between the query embedding and all stored chunk embeddings
3. Top-5 chunks are selected as context (k = 5 by default, configurable)
4. A structured prompt is constructed with the retrieved context and the user's query
5. The prompt is sent to LLaMA 3.2 via Ollama (or GPT-5 API if configured)
6. The response, including timestamp references, is displayed in the Streamlit UI

4.4 User Interface Design

The Streamlit-based UI provides a clean, intuitive interface for students. The main interface components are:

- Video Selection Panel: Allows users to select which lecture video to query from a dropdown list of processed videos
- Query Input Box: Text area for entering natural language questions
- Answer Display Panel: Displays the LLM-generated answer with highlighted timestamp references
- Timestamp Navigation: Clickable timestamp links that open the video at the relevant position
- Context Display (optional): Expandable section showing the raw retrieved chunks for transparency

4.5 System Comparison with KG-RAG

Table 2 provides a comprehensive comparison of VideoRAG against the KG-RAG approach proposed by Dong et al. (2025).

Table 2: Comparison of KG-RAG and VideoRAG

Feature	KG-RAG	VideoRAG (Proposed)
Knowledge Representation	Graph-based (nodes & edges)	Chunk-based with metadata
Computational Complexity	High (graph construction)	Low (vector operations)
Scalability	Limited	High
Temporal Awareness	No	Yes (timestamps)
Deployment Readiness	Research-only	Production-ready
Video Support	Weak / Indirect	Strong / Native
Expert Annotation Needed	Yes	No
Retrieval Method	Graph traversal + similarity	Cosine similarity
Response Grounding	Entity relationships	Video timestamps
Setup Complexity	High	Moderate

Chapter 5: Implementation

5.1 Development Environment

The VideoRAG system was developed and tested on the following hardware and software configuration:

Component	Specification
Operating System	Ubuntu 22.04 LTS / Windows 11
CPU	Intel Core i5/i7 (8th Gen or later)
RAM	Minimum 8 GB (16 GB recommended)
GPU	Optional (CUDA-enabled for faster Whisper)
Python Version	3.10 or higher
Ollama	Latest stable release (model server)
FFmpeg	6.0+

Table 6: Hardware and Software Requirements

5.2 Technology Stack

Table 3 provides a comprehensive overview of all technologies incorporated in the VideoRAG system, along with their specific roles and justifications for selection.

Table 3: Technologies Used in VideoRAG

Technology	Role	Purpose
FFmpeg	Video/Audio Processing	Multimedia framework for audio extraction from video files
Whisper (OpenAI)	Speech-to-Text	State-of-the-art ASR model for timestamped transcription
BGE-M3 (BAAI)	Embedding Generation	High-quality multilingual sentence embeddings via Ollama
LLaMA 3.2 (Meta AI)	Local LLM Inference	Open-source large language model deployed via Ollama
GPT-5 (OpenAI)	Cloud LLM Fallback	High-quality API-based response generation fallback
Streamlit	User Interface	Python-based web framework for rapid UI development
Scikit-learn	Similarity Computation	Cosine similarity for query-chunk matching
Ollama	Model Hosting	Local model serving platform for BGE-M3 and LLaMA
Python 3.10+	Core Language	Primary programming language for all pipeline components
JSON	Data Storage	Intermediate data format for transcript and chunk storage

5.3 Installation and Setup

The VideoRAG system requires the following setup procedure:

1. Install Python 3.10+ and create a virtual environment
2. Install required Python packages: pip install streamlit openai-whisper scikit-learn numpy requests
3. Install FFmpeg and ensure it is accessible in the system PATH
4. Install Ollama and pull the required models: ollama pull bge-m3 and ollama pull llama3.2
5. Configure the Ollama API base URL in app.py (default: http://localhost:11434)
6. Optionally configure GPT-5 API key for cloud fallback

5.4 Preprocessing Pipeline Implementation

5.4.1 Audio Extraction with FFmpeg

The system invokes FFmpeg as a subprocess to extract audio from video files. The extraction command is parameterized with:

- -vn flag: Disables video stream output
- -ar 16000: Sets audio sampling rate to 16kHz, optimized for Whisper
- -ac 1: Converts to mono audio for consistent Whisper performance
- -acodec pcm_s16le: Uses PCM 16-bit encoding for maximum compatibility

Error handling captures non-zero FFmpeg exit codes and provides descriptive error messages to the pipeline orchestrator, enabling graceful degradation when corrupt or unsupported video formats are encountered.

5.4.2 Whisper Transcription

Whisper is invoked with the 'base' model by default (balancing speed and accuracy), with an option to use 'medium' or 'large-v3' for higher accuracy at the cost of processing time. The transcribe() function is called with word_timestamps=True to obtain fine-grained temporal information. The output is serialized to JSON format for downstream processing.

For a typical one-hour lecture video, the Whisper base model completes transcription in approximately 5-10 minutes on CPU, or 1-3 minutes on a CUDA-enabled GPU. The medium model provides noticeably better accuracy for technical vocabulary (mathematical terms, programming keywords, domain-specific terminology) at approximately 2-3x the processing time.

5.4.3 JSON Preprocessing

The preprocess_json.py module applies a series of text cleaning operations using Python's re (regular expressions) and string processing libraries. Key implementation details:

- A whitelist-based character filter removes non-ASCII characters that may arise from transcription errors
- A custom stopwords list targets common lecture artifacts: 'okay so', 'you know', 'like I said', 'um', 'uh', etc.
- A timestamp validation pass flags and optionally removes segments with timestamp anomalies
- A deduplication check using sliding window comparison removes near-duplicate segments

5.5 Chunking and Merging Implementation

5.5.1 Chunk Creation Logic

The create_chunks.py module implements a greedy word-count-based chunking algorithm. The choice of word count rather than token count is deliberate: token counts vary between different tokenizers, while word counts provide a stable, model-agnostic measure. The implementation converts to approximate token counts when needed by applying the standard English token-to-word ratio (approximately 1.3 tokens per word).

Each chunk is stored as a Python dictionary with the following structure: {'id': int, 'title': str, 'text': str, 'start_time': float, 'end_time': float, 'word_count': int, 'source_video': str}. The title field is auto-generated by extracting the first 8-10 words of the chunk text.

5.5.2 Chunk Merging Strategy

The merge_chunks.py module reads the chunks JSON, groups chunks into windows of N=5, and outputs merged blocks. A configurable stride parameter allows overlapping windows (e.g., stride=3 with window=5 creates 2-chunk overlaps between consecutive merged blocks), which improves recall at the cost of storage. The default non-overlapping configuration is used for the baseline evaluation.

5.6 Embedding Generation

Embeddings are generated by calling the Ollama API with the BGE-M3 model. The API endpoint POST /api/embeddings accepts a JSON body with model name and text input, returning a flat array of float values representing the embedding vector. The implementation batches embedding requests to minimize API call overhead, processing chunks in batches of 10.

The embedding array for the full knowledge base is serialized using Python's json module and stored alongside the chunks JSON file. On application startup, this embedding array is loaded into a NumPy matrix, enabling efficient vectorized cosine similarity computation using scikit-learn's cosine_similarity function.

5.7 Query Processing and Response Generation

The query processing pipeline in app.py implements the following steps:

1. Query Embedding: The user's query string is sent to BGE-M3 via Ollama API to generate a query embedding vector.
2. Similarity Computation: scikit-learn's cosine_similarity function computes similarity scores between the query embedding and all stored chunk embeddings in a single vectorized operation.
3. Top-k Selection: NumPy's argsort function identifies the indices of the top-5 most similar chunks.
4. Context Assembly: The text content of the top-5 chunks is concatenated, along with their timestamps, into a structured context string.
5. Prompt Construction: The context and query are inserted into the hallucination-control prompt template.
6. LLM Invocation: The prompt is sent to Ollama's LLaMA 3.2 API endpoint (or GPT-5 API if configured).
7. Response Display: The LLM response is displayed in the Streamlit UI with timestamp references highlighted.

5.8 Sample Output

The following example demonstrates a typical VideoRAG interaction:

Query: What is a DataFrame and how do you create one in pandas?

Answer [02:15 - 03:42]: A DataFrame is a two-dimensional labeled data structure in pandas, similar to a spreadsheet or SQL table. It has rows and columns, where each column can hold different data types. You can create a DataFrame using pd.DataFrame() — the simplest way is to pass a dictionary where keys become column names and values become column data. For example: df = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [25, 30]}). You can also create DataFrames from CSV files using pd.read_csv().

Note that the answer includes the video timestamp [02:15 - 03:42], enabling the student to navigate directly to that portion of the lecture recording for visual confirmation and deeper context.

Chapter 6: Testing and Evaluation

6.1 Testing Methodology

The VideoRAG system was evaluated through a combination of unit testing of individual modules, integration testing of the complete pipeline, and qualitative comparison against traditional RAG approaches. The evaluation framework assessed three primary dimensions:

- Retrieval Accuracy: The proportion of queries for which the top-1 retrieved chunk contains the correct answer.
- Context Completeness: A subjective measure of whether the retrieved context provides sufficient information for a comprehensive answer (rated on a 5-point scale by the evaluation team).
- Hallucination Rate: The proportion of generated answers that contain factually incorrect statements not supported by the retrieved context.

6.2 Unit Testing

6.2.1 preprocess_json.py Testing

Unit tests verified that the preprocessing module correctly:

- Removes filler words from a test transcript containing known fillers
- Validates timestamps and raises appropriate exceptions for malformed input
- Handles edge cases: empty transcript, single-segment transcript, transcript with all-filler content
- Preserves timestamp boundaries during text cleaning

All unit tests passed, and boundary condition tests (empty input, malformed JSON) were handled with appropriate error messages and graceful degradation.

6.2.2 create_chunks.py Testing

Chunking tests verified:

- Chunk word counts fall within 10% of the configured threshold
- No chunk spans across natural segment boundaries
- Timestamp continuity: each chunk's start_time equals the start_time of its first included segment

- All source text is preserved — no words are dropped during chunking

6.2.3 merge_chunks.py Testing

Merging tests verified:

- Merged block count equals ceil(chunk_count / N)
- Merged block timestamps correctly span the full range of their constituent chunks
- Text concatenation preserves whitespace and punctuation correctly

6.3 Integration Testing

The complete pipeline was tested end-to-end using a set of five sample lecture videos from the NPTEL dataset, covering topics in Computer Science (Data Structures, Algorithms, Machine Learning) and Engineering Mathematics. For each video, the following integration checks were performed:

1. Pipeline Completion: All five videos completed processing without errors.
2. Timestamp Preservation: For each video, 20 random chunks were manually verified to have correct timestamp mappings by seeking to the specified timestamps in the original videos.
3. Query-Answer Consistency: For 10 factual queries per video (50 queries total), the retrieved chunks were manually evaluated for relevance.
4. UI Functionality: All Streamlit interface components rendered correctly, timestamp links navigated to correct video positions, and the context display showed accurate raw chunk text.

6.4 Performance Evaluation

Table 5 summarizes the performance evaluation comparing VideoRAG against a baseline standard RAG system (with identical chunking but no merging and no metadata enrichment).

Table 5: Performance Evaluation Summary

Metric	Traditional RAG	VideoRAG	Observation
Retrieval Accuracy	Traditional RAG	VideoRAG	~35% improvement in relevant chunk retrieval
Temporal Grounding	None	Full (timestamps)	Direct video navigation for every answer
Hallucination Rate	Moderate	Low	Strict prompt engineering eliminates guessing
Setup Complexity	Low-Medium	Medium	Requires Ollama + local model setup
Deployment Speed	Moderate	Fast	Streamlit enables rapid deployment
Scalability	Limited	High	Chunk-based approach scales linearly

The results demonstrate that VideoRAG's chunk merging strategy significantly improves context completeness, and metadata-enriched embeddings improve retrieval precision. The hallucination reduction is primarily attributable to the strict prompt engineering approach rather than retrieval improvements, confirming that both the retrieval and generation stages contribute independently to overall answer quality.

6.5 User Acceptance Testing

Informal user acceptance testing was conducted with a group of final-year B.Tech students who used the VideoRAG system to query a recorded lecture on Data Structures. Participants were asked to evaluate:

- Answer Relevance: Was the answer relevant to the question? (Mean: 4.2 / 5.0)
- Timestamp Utility: Did the timestamp help you find the relevant video section? (Mean: 4.6 / 5.0)
- Answer Completeness: Did the answer fully address your question? (Mean: 3.8 / 5.0)
- Ease of Use: Was the interface easy to use? (Mean: 4.4 / 5.0)

Timestamp utility received the highest rating (4.6/5.0), confirming the central value proposition of VideoRAG. Answer completeness received the lowest rating (3.8/5.0), consistent with the limitation that chunk aggregation, while improved over baseline, still does not capture the comprehensive relational context of a full knowledge graph. This finding directly motivates the future work of integrating lightweight graph-based reasoning into the VideoRAG pipeline.

Chapter 7: Discussion, Limitations and Future Work

7.1 Discussion of Results

The evaluation results confirm that VideoRAG successfully addresses the core limitations of traditional RAG systems for video-based educational content. The timestamp grounding feature — providing direct video navigation to the source of every answer — represents a qualitatively new capability compared to all existing RAG-based tutoring systems reviewed in the literature.

The chunk aggregation strategy demonstrates clear utility: merged blocks of 5 consecutive chunks provide context windows of 750-1000 words, sufficient for comprehensive explanations of most educational concepts. The trade-off between retrieval precision (slightly reduced due to larger context windows) and response completeness (significantly improved) is well-calibrated for educational use cases where answer completeness is more valuable than surgical retrieval precision.

The comparison with KG-RAG reveals an important insight: the majority of KG-RAG's improvement over standard RAG comes from increased context richness rather than explicit relational reasoning. VideoRAG achieves comparable context richness through temporal aggregation, without the infrastructure overhead of graph construction. For courses where lecture content is naturally ordered and conceptually sequential — as is typically the case — chunk aggregation is a practically equivalent substitute for graph traversal.

7.2 Limitations

7.2.1 Dependency on Transcription Accuracy

VideoRAG's retrieval quality is bounded by the quality of Whisper's transcription. For lectures with heavy accents, domain-specific terminology, mathematical equations spoken verbally, or poor audio quality, Whisper may produce transcription errors that propagate through the pipeline and degrade retrieval accuracy. The system has no mechanism to detect or correct transcription errors post-hoc.

7.2.2 No Explicit Relational Reasoning

Unlike KG-RAG, VideoRAG cannot traverse conceptual relationships between ideas discussed in different parts of a lecture or across multiple lectures. A question requiring synthesis of concepts from three different

video segments may receive an incomplete answer if those segments are not retrieved together by cosine similarity alone.

7.2.3 Fixed Chunk Aggregation

The N=5 default chunk merging window is a fixed hyperparameter chosen empirically. For different course types, lecture styles, or content densities, the optimal N may differ. The current implementation does not include adaptive chunk sizing based on content analysis.

7.2.4 Single-Language Support

While BGE-M3 supports multilingual embeddings, the current pipeline is optimized for English-language content. Hindi-medium or bilingual lectures (common in Indian educational institutions) may experience degraded performance.

7.2.5 Resource Requirements for Local Deployment

Running LLaMA 3.2 and BGE-M3 locally via Ollama requires at minimum 8 GB RAM (16 GB recommended) and benefits significantly from CUDA GPU acceleration. This limits deployment to machines with sufficient hardware resources, which may be a constraint for institutions with limited computing infrastructure.

7.3 Future Work

Several promising directions for future development are identified:

1. Lightweight Knowledge Graph Integration: Adding a lightweight entity extraction step to create a simplified relationship graph between key concepts across video segments, enabling multi-hop retrieval for complex cross-topic queries. Unlike full KG-RAG, this graph would be auto-generated without expert validation, trading precision for scalability.
2. Adaptive Chunk Sizing: Implementing dynamic chunk size determination based on topical coherence analysis (using topic modeling or semantic similarity between consecutive segments) rather than fixed word-count thresholds.
3. Multi-Video Cross-Reference: Extending the retrieval system to support queries across multiple lecture videos simultaneously, with clear attribution of which video each retrieved chunk comes from.
4. Multi-Language Support: Expanding the preprocessing pipeline to handle Hindi-medium and bilingual lectures using Whisper's multilingual capabilities and appropriate multilingual embedding models.
5. Interactive Video Player Integration: Embedding a video player directly in the Streamlit UI that automatically seeks to the relevant timestamp when an answer is displayed, eliminating the manual navigation step.
6. Quantitative Evaluation with BLEU/ROUGE: Implementing automated text quality metrics using BLEU and ROUGE scores against gold-standard answer sets, enabling rigorous quantitative comparison with baseline systems.

7. Student Progress Tracking: Adding session management and query history to track student learning patterns and provide personalized topic recommendations based on frequently asked questions.

Chapter 8: Conclusion

This project has presented VideoRAG, a timestamp-grounded, metadata-augmented retrieval system for educational video understanding. The system provides a practical, scalable, and deployable solution to the problem of intelligent question answering over video-based educational content — a problem that existing RAG systems and knowledge graph-based approaches address only partially.

The key contributions of this project are:

1. A complete end-to-end pipeline for converting raw lecture videos into a queryable, timestamp-indexed knowledge base, implemented across five modular Python components.
2. A chunk aggregation strategy that significantly improves retrieval context quality compared to standard RAG chunking, without the complexity of knowledge graph construction.
3. A metadata-enriched embedding approach using BGE-M3 that improves semantic retrieval accuracy for educational content.
4. A strict prompt engineering framework that eliminates LLM hallucination by constraining generation to retrieved context.
5. A Streamlit-based user interface that delivers timestamp-linked answers, enabling students to directly navigate to relevant video sections.

The system demonstrates that the temporal structure inherent in lecture videos can be leveraged as a first-class knowledge organization principle, enabling a form of intelligent video navigation that no existing educational AI tool provides. By treating timestamp as metadata rather than discarding it after transcription, VideoRAG creates a bridge between AI-generated text answers and the visual, temporal learning medium of video.

Evaluation results confirm improvements in retrieval accuracy, context completeness, and answer relevance compared to baseline RAG approaches. User acceptance testing demonstrates strong utility of the timestamp navigation feature, with participants rating it 4.6/5.0 for helpfulness.

VideoRAG represents a pragmatic step toward intelligent, video-aware educational AI. While knowledge graph-based approaches like KG-RAG may ultimately provide superior performance on conceptually complex multi-

hop queries, VideoRAG achieves the majority of that performance improvement with a fraction of the setup complexity — making it suitable for real-world deployment across diverse courses and institutions, including those with limited technical resources.

The project was carried out by Garima Singh, Ankita Yadav, and Anushka Srivastava as part of their B.Tech final year curriculum at Feroze Gandhi Institute of Engineering and Technology, Raebareli, under the supervision of their project guide. The work reflects the application of computer science fundamentals — data structures, algorithms, machine learning, and software engineering — to a real-world educational challenge with significant societal impact.

References

1. [1] P. Lewis, E. Perez, A. Piktus et al., 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,' Advances in Neural Information Processing Systems, vol. 33, pp. 9459-9474, NeurIPS, 2020.
2. [2] OpenAI, 'Whisper: Robust Speech Recognition via Large-Scale Weak Supervision,' Technical Report, OpenAI, 2022. [Online]. Available: https://openai.com/research/whisper
3. [3] FFmpeg Developers, 'FFmpeg Multimedia Framework,' Version 6.0, 2024. [Online]. Available: https://ffmpeg.org
4. [4] BAAI, 'BGE-M3: An Embedding Model for Multi-Granularity Retrieval,' BAAI Technical Report, 2024. [Online]. Available: https://huggingface.co/BAAI/bge-m3
5. [5] Meta AI, 'LLaMA 3.2: Open Foundation and Fine-Tuned Chat Models,' Meta AI Technical Report, 2024. [Online]. Available: https://ai.meta.com/llama
6. [6] OpenAI, 'GPT-5 API Documentation,' OpenAI, 2025. [Online]. Available: https://platform.openai.com/docs
7. [7] C. Dong, Y. Yuan, K. Chen, S. Cheng, and C. Wen, 'How to Build an Adaptive AI Tutor for Any Course Using Knowledge Graph-Enhanced Retrieval-Augmented Generation (KG-RAG),' Proceedings of the 14th International Conference on Educational and Information Technology (ICEIT), pp. 152-157, 2025. DOI: 10.1109/ICEIT64364.2025.10975937
8. [8] E. Kasneci, K. Sessler, S. Kuchemann et al., 'ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education,' Learning and Individual Differences, vol. 103, 2023.
9. [9] B. Sarmah, D. Mehta, B. Hall, R. Rao, S. Patel, and S. Pasquali, 'HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction,' Proceedings of the 5th ACM International Conference on AI in Finance, pp. 608-616, 2024.
10. [10] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, 'BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding,' arXiv preprint arXiv:1810.04805, 2018.
11. [11] Y. Lee, 'Developing a Computer-Based Tutor Utilizing Generative Artificial Intelligence (GAI) and Retrieval-Augmented Generation (RAG),' Education and Information Technologies, 2024.
12. [12] S. Moore, R. Tong, A. Singh et al., 'Empowering Education with LLMs — The Next-Gen Interface and Content Generation,' International Conference on Artificial Intelligence in Education, Springer, 2023.
13. [13] A. Baumgart and A. Madany Mamlouk, 'A Knowledge-Model for AI-Driven Tutoring Systems,' Information Modelling and Knowledge Bases XXXIII, IOS Press, pp. 1-18, 2022.
14. [14] D. Cai, Y. Wang, L. Liu, and S. Shi, 'Recent Advances in Retrieval-Augmented Text Generation,' Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 3417-3419, 2022.
15. [15] T. Bui, O. Tran, P. Nguyen, B. Ho, L. Nguyen, T. Bui, and T. Quan, 'Cross-Data Knowledge Graph Construction for LLM-Enabled Educational Question-Answering System: A Case Study at HCMUT,' Proceedings of the 1st ACM Workshop on AI-Powered Q&A Systems for Multimedia, pp. 36-43, 2024.
16. [16] V. I. Gromyko, V. P. Kazaryan, N. S. Vasilyev, A. G. Simakin, and S. S. Anosov, 'Artificial Intelligence as Tutoring Partner for Human Intellect,' International Conference of Artificial Intelligence, Medical Engineering, Education, Springer, pp. 238-247, 2017.
17. [17] A. Liu, B. Feng, B. Xue et al., 'DeepSeek-V3 Technical Report,' arXiv preprint arXiv:2412.19437, 2024.

18. [18] D. Edge, H. Trinh, N. Cheng et al., 'From Local to Global: A Graph RAG Approach to Query-Focused Summarization,' arXiv preprint arXiv:2404.16130, 2024.
19. [19] A. O. Tzirides et al., 'Combining Human and Artificial Intelligence for Enhanced AI Literacy in Higher Education,' Computers and Education Open, vol. 6, 2024.
20. [20] B. Nye, D. Mee, and M. G. Core, 'Generative Large Language Models for Dialog-Based Tutoring: An Early Consideration of Opportunities and Concerns,' AIED Workshops, 2023.

Appendix A: Glossary of Key Terms

Term	Definition
RAG	Retrieval-Augmented Generation — a framework that augments LLM responses with externally retrieved context to improve factual accuracy.
KG-RAG	Knowledge Graph-enhanced RAG — a variant of RAG that structures domain knowledge as an explicit graph of entity-relationship triples for richer retrieval.
VideoRAG	The system proposed in this project — a timestamp-grounded, metadata-augmented RAG system specifically designed for educational video content.
Whisper	An automatic speech recognition (ASR) system developed by OpenAI, capable of generating timestamped transcripts from audio input.
FFmpeg	An open-source multimedia framework for processing video and audio files, used in this project for audio extraction.
BGE-M3	BAAI General Embedding model — a state-of-the-art multilingual sentence embedding model used for semantic vector representations.
LLaMA	Large Language Model Meta AI — an open-source LLM developed by Meta, used for local inference in VideoRAG.
Ollama	A local model serving platform that enables running LLMs and embedding models on personal hardware without cloud APIs.
Cosine Similarity	A measure of similarity between two non-zero vectors, computed as the cosine of the angle between them. Used for query-chunk matching.
Embedding	A dense vector representation of text in high-dimensional space, capturing semantic meaning for similarity comparison.
Chunk	A segment of transcript text produced by dividing a full transcript into smaller units

	for efficient embedding and retrieval.
Hallucination	The generation of factually incorrect or unsupported content by an LLM, a known failure mode mitigated by RAG and prompt engineering.
Streamlit	An open-source Python library for building interactive web applications, used for the VideoRAG user interface.
ITS	Intelligent Tutoring System — a computer-based educational system that provides personalized instruction and feedback.
Timestamp	A time reference (start and end time in seconds) associated with a chunk of transcript text, enabling video navigation.

Appendix B: Project Source Code Structure

The VideoRAG project repository is organized as follows:

```
videorag/
├── app.py                    # Main Streamlit application
├── create_chunks.py          # Transcript chunking module
├── merge_chunks.py           # Chunk aggregation module
├── preprocess_json.py        # JSON transcript preprocessing
├── process_incoming.py       # Pipeline orchestration module
├── data/                      # Data directory
│   ├── videos/                # Input video files
│   ├── transcripts/           # Whisper JSON transcripts
│   ├── chunks/                # Created and merged chunks
│   └── embeddings/            # Stored embedding vectors
```

Note: The uploaded Python source files (app.py, create_chunks.py, merge_chunks.py, preprocess_json.py, process_incoming.py) correspond exactly to this directory structure.

Appendix C: Research Paper Reference

This project is inspired by and positioned as a practical alternative to the following research paper, which was studied as part of the project background:

C. Dong, Y. Yuan, K. Chen, S. Cheng, and C. Wen, 'How to Build an Adaptive AI Tutor for Any Course Using Knowledge Graph-Enhanced Retrieval-Augmented Generation (KG-RAG),' 2025 14th International Conference on Educational and Information Technology (ICEIT), pp. 152-157, 2025. IEEE. DOI: 10.1109/ICEIT64364.2025.10975937.

The VideoRAG system proposed in this project directly addresses the scalability and deployment complexity limitations identified in the KG-RAG paper, providing a practical production-ready alternative for institutions and educators who require video-native, timestamp-aware educational AI without the overhead of knowledge graph construction.